A Project by EHS

Software Department

VistA Code Debugging

Version 0.3
2, Oct 2013

Document Information

Document Name	Vista Code Debugging
Electronic File Name	VistA_Code_Debugging (3).odt
Version	0.1
Status	Reviewed
Date Issued	5, Dec 2013
Authors Name	Heba Hammouri and Sinan Imrash
Owner	Electronic Health Solutions
Reviewed By	Mohammad Abu Dayyeh and Bayan Hashem
Approved By	
Approval Date	

Revision History

Date	Revision	Description Of Change	Change Owner
2, Oct 2013	0.1	Document written	Heba Hammouri and Sinan Imrash
3, Oct 2013	0.2	Document reviewed	Mohammad Abu Dayyeh
5, Dec 2013	0.3	Final review and minor fixes	Bayan Hashem

Abbreviations and Acronyms

Abbreviation/Acronym	Expansion
GUI	Graphical User Interface
VistA	Veterans Information Systems and Technology Architecture
EHS	Electronic Health Solutions
API	Application Programming Interface
CPRS	Computerized Patient Record System
M	MUMPS programming language
MUMPS	Massachusetts General Hospital Utility Multi-programming System
GT.M	Greystone Technology M
RPC	Remote Procedure Call

Table of Contents

1. Executive Summery

This Demonstrate the debugging process that the programmer need to trace a code by stopping the execution of an option to see what the system did, understand how exactly the option work, or find an error. This facilitates to the programmer trace the code, or change it to meet our goals to make the customers satisfied.

2. Introduction

GT.M runs on many platforms, regardless of the difference between them, also it keeps the traditional features of M coding. In addition, it enables the user to create, compile, execute, edit, and debug M codes.

In this document we will present one of the facilities that GT.M provide; **Debugging**. It's your way to run and understand the code, set breakpoints, and find errors, print and fix them. It's also enable you to execute the code line by line by using a specific commands.

We will explain the following types of debugging:

1. Direct Mode Debug.
2. RPC Listener.
3. CPRS RPCs' Debugging.
4. TMGIDE Debugger.

1.1 Prerequisites and Dependencies

The following are a prerequisite for a programmer user:

1. Sufficient knowledge of M language, programming skills.
2. Familiar with GT.M environment.
3. FileMan and Kernel functionalities.
4. CPRS GUI application access.
5. Assign “XUPROG” key to allow CPRS RPC's Debugging.

Direct Mode Debug

This section introduces the first way to debug a code (run lines of code, print code, show stack, …, etc) and helps you to do debugging using number of important commands.

2.1 Main Commands

this section presents the most important commands that you will use during debug process, and the commands are:

- **ZBREAK (ZB): S**et a temporary break point during debugging.
- **ZCONTINUE (ZC):** Resume the routine execution from the break point.
- **ZWRITE (ZWR):** Print the current local variables with their values.
- **ZPRINT (ZP):** Print the M code lines belonged to its argument.
- **ZSHOW (ZSH):** Print the current stack status (information about GT.M environment).
- **ZSTEP (ZST):** Execute the routine line by line and give you the next M line to execute.
- **ZSTEP INTO:** Move the control to the routine(or any DO command) inside the parent routine to debug it.
- **ZSTEP OUTOF:** Resume the execution until the first QUIT command it finds, and move the control to the next level in the stack.

2.1 Main Special Variables

This section introduces the important special variables that you will use during debugging, and the variables are:

- **$ZPOSITION ($ZPOS)**: Contains a string value of current line reference:

  ([functionName][+lineNumber]^[routineName]).
- **$ZSTEP ($ZST):** Contains string value that the ZSTEP will do when calling it.

2.3 How To Debug?

After reading this section you will be able to debug VistA code. Just log into Vista and do the following:

1. Establish a break point to start debugging by using **ZBREAK** command

   Example:

   >ZBREAK TEST+3^KJOTEST

   Where KJOTEST is the routine name and TEST is the function name. The breakpoint on line #3.
2. Re-set the $ZSTEP value to print the current line (M code)

   >SET $ZSTEP=”ZPRINT @$ZPOSITION BREAK”
3. Run the routine by DO command to start debugging

   >DO TEST^KJOTEST

The GT.M will indicate that a break point exist

4. Start debugging by using ZSTEP & ZSTEP INTO commands, that will execute the current line and display the next line to execute:

   >ZSTEP

5. ZWRITE command is used to display the current local variables and their current values.

   >ZWRITE

6. ZSTEP INTO is used to debug invoked routine (or the “FOR” command) explicitly that exist in the current line of code

   >ZSTEP INTO

7. ZSHOW command is used to display information about the M environment (Show everything in the stack).

   >ZSHOW

8. ZCONTINUE command is used to complete the execution of the code from the last breakpoint.

   >ZCONTINUE

3. RPC Listener

"RPC broker is a software which establish a connection from a client workstation to a VistA M Server, run RPCs on the VistA M Server, and return data to the client workstation. The VistA M Server continuously runs an RPC Broker listener process whose purpose is to establish connections with clients, when the listener process receives a connection request from a client, it produce a separate handler process, which then handles all communications with the client. Once connected, the client can execute Remote Procedure Calls on the VistA M Server".

4.1 Listening Configurations

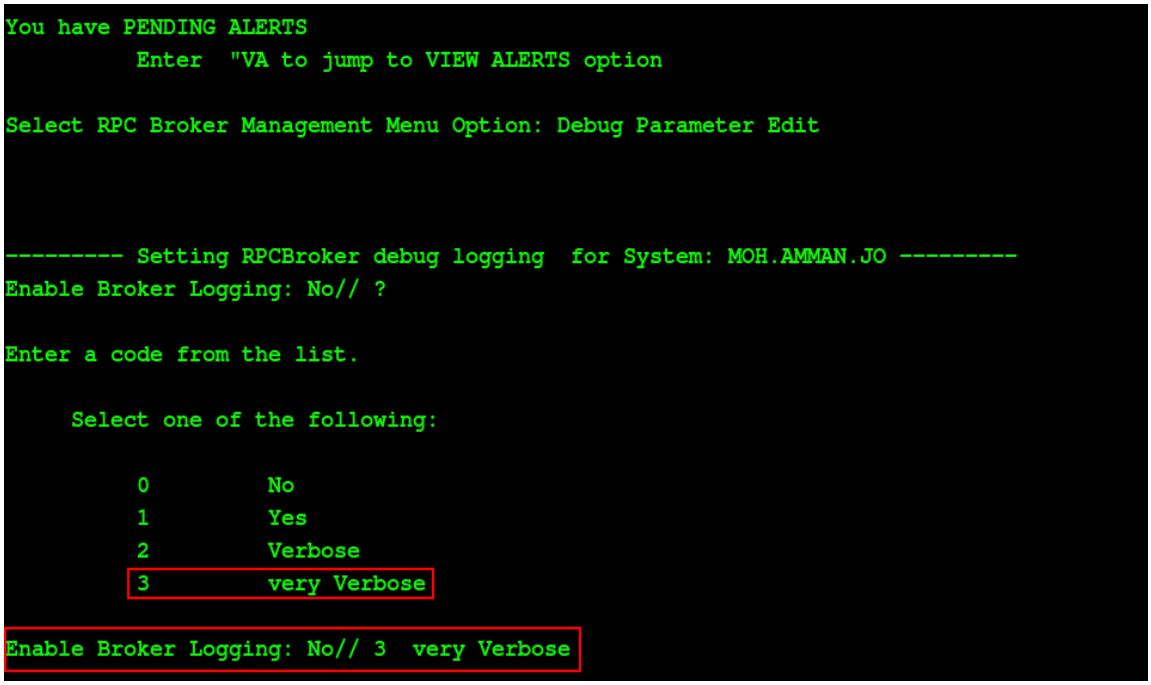
The following steps are required to start listening properly within "RPC Broker Management Menu [XWB MENU]" Menu.

- Edit the listener configurations by "RPC Listener Edit [XWB LISTENER EDIT]" option

  The following example sets a listener for **sw-dev** with **9230** port:

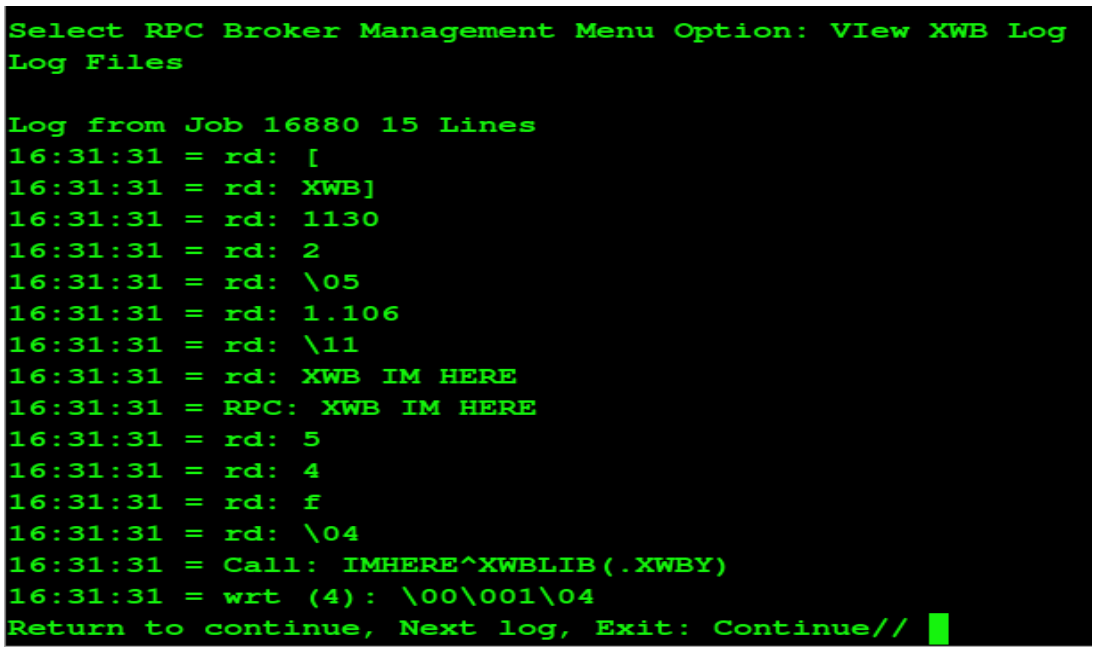
```
Select RPC BROKER SITE PARAMETERS DOMAIN NAME: BETA.VISTA-OFFICE.ORG
Select BOX-VOLUME PAIR: EHR:sw-dev//
  BOX-VOLUME PAIR: EHR:sw-dev//
  Select PORT: 9230//
    PORT: 9230//
    STATUS: STOPPED//
    TYPE OF LISTENER: New Style//
    CONTROLLED BY LISTENER STARTER: YES//
```

- Set **Very Verbose** mode by “Debug Parameter Edit [XWB DEBUG EDIT]” option.



- Start the listener by “Start All RPC Broker Listeners [XWB LISTENER STARTER]” option.
- Clear the log by “Clear XWB Log Files [XWB LOG CLEAR]” option.
- View the log from the “View XWB Log [XWB LOG VIEW]” option (find the screen shot below)
  For more details refer to the following link:

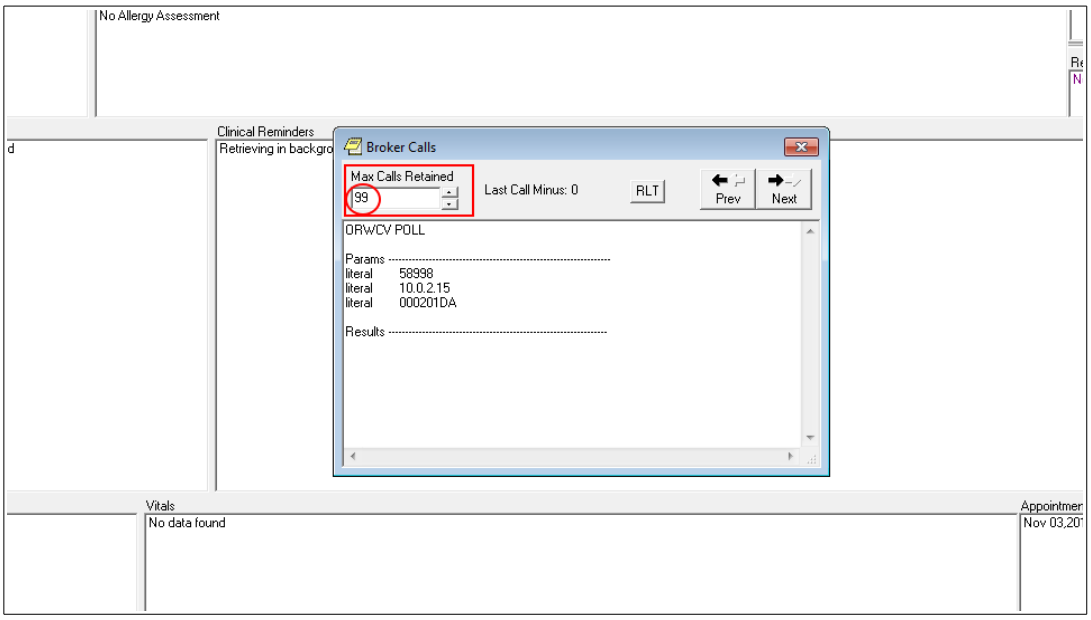
  www4.va.gov/vdl/documents/Infrastructure/Remote_Proc_Call_Broker_(RPC)/xwb1_1p47systems_management_guide.pdf

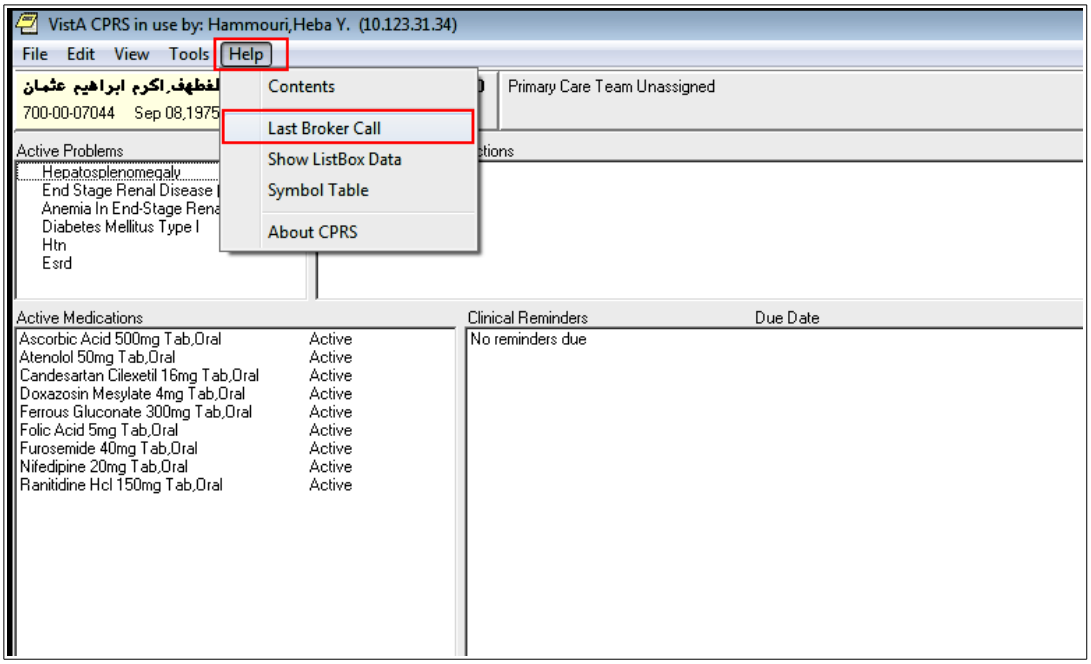


4. CPRS RPCs' Debugging

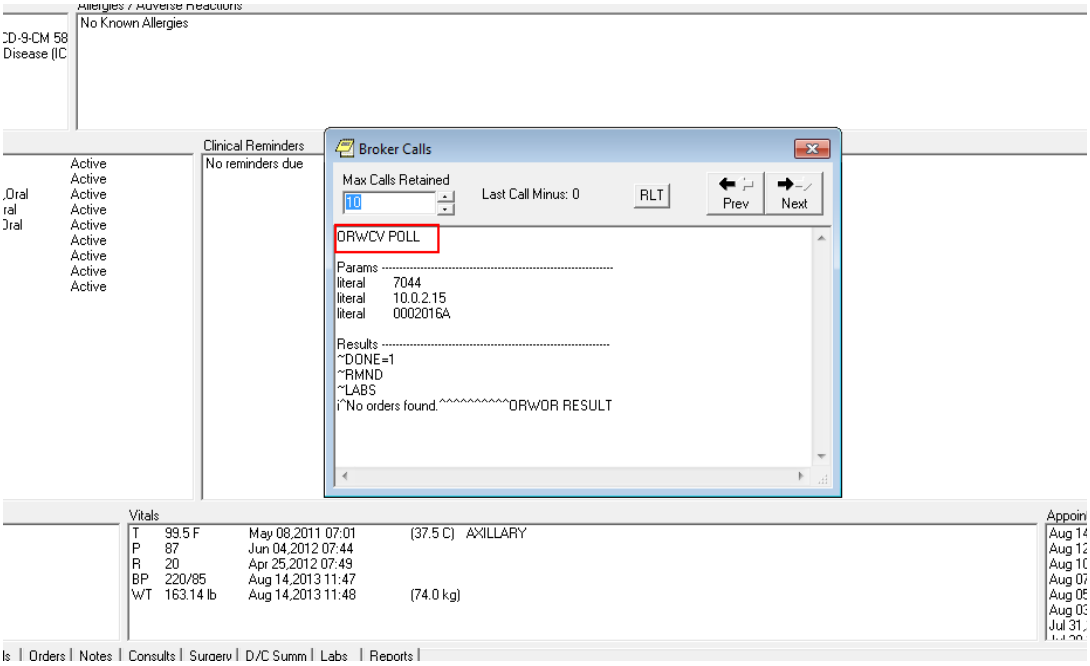
This section introduces the way to debug using CPRS GUI. The following steps are required to start CPRS RPC's Debugging properly:

- Run CPRS and select a patient.
- Increase the “Max Calls Retained” to 99 by selecting Help menu→Last Broker Call as shown below.





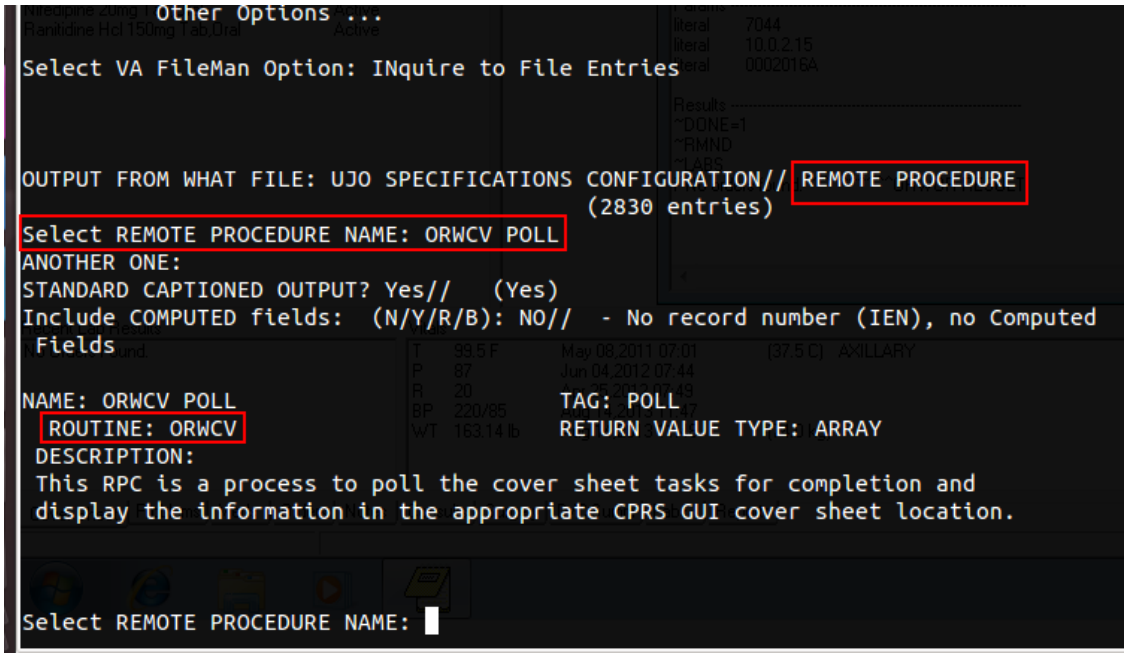
- List the RPC's that is used in CPRS by selecting Help menu→Last Broker Call.



The Prev & Next buttons enable you to navigate between all calls from the last to the first.

- Since you get the RPC name (as mentioned above), then you can inquire that RPC in VistA from REMOTE PROCEDURE file #8994 (and get the routine name to start debugging within the routine) as the following:

  VA FileMan [DIUSER]" option → "Inquire to File Entries [DIINQUIRE]" option → Enter the RPC name (find the screen shot below)



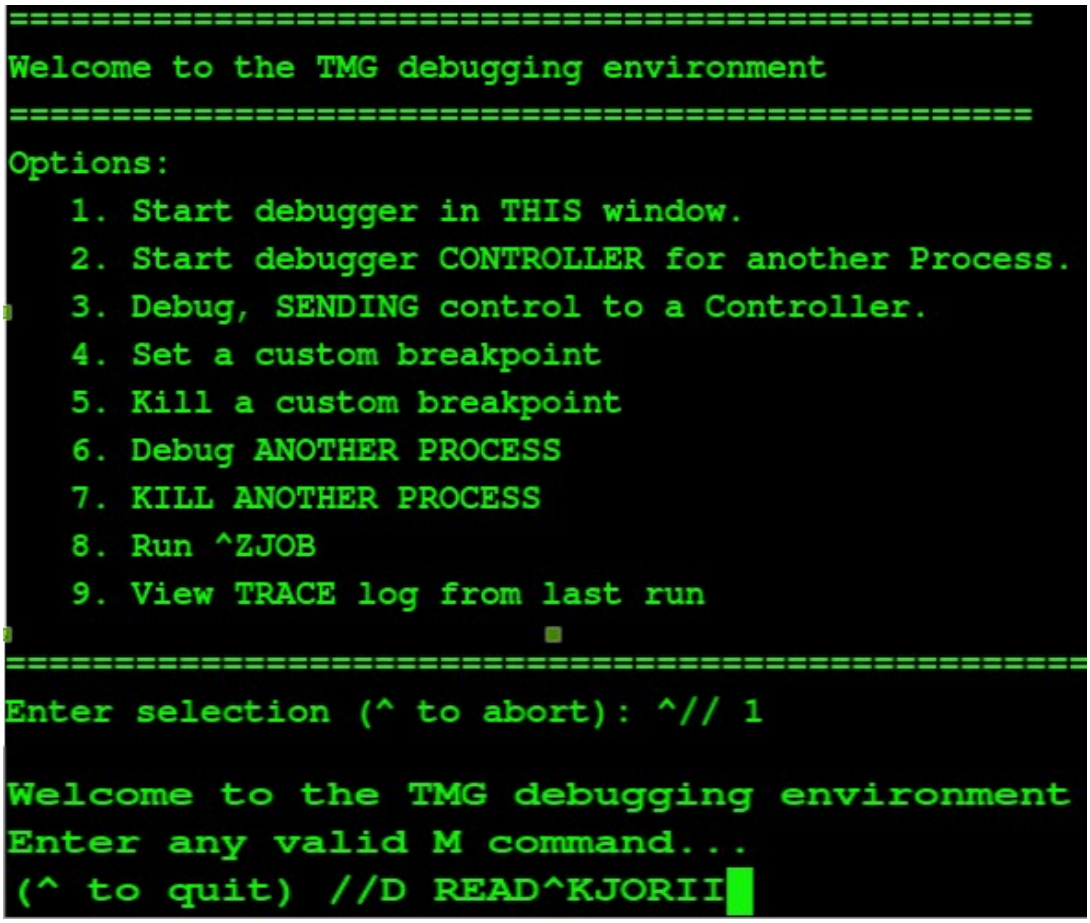
5. TMGIDE Debugger

TMGIDE is a user friendly debugging/tracing tool for GT.M, this section learns you how to use it.

5.1 Start Debugger

The following steps are required to start TMGIDE debugging:

- Set the DUZ variable (ex: GT.M>SET DUZ=8)
- Call ^XUP (GTM>DO ^XUP)
- Call the TMGIDE (GTM>DO ^TMGIDE) to start tool functionality
- Enter “Start debugger in THIS window” value #1 in “Enter selection (^ to abort): ^//” prompt
- Run the required routine (find the example below)



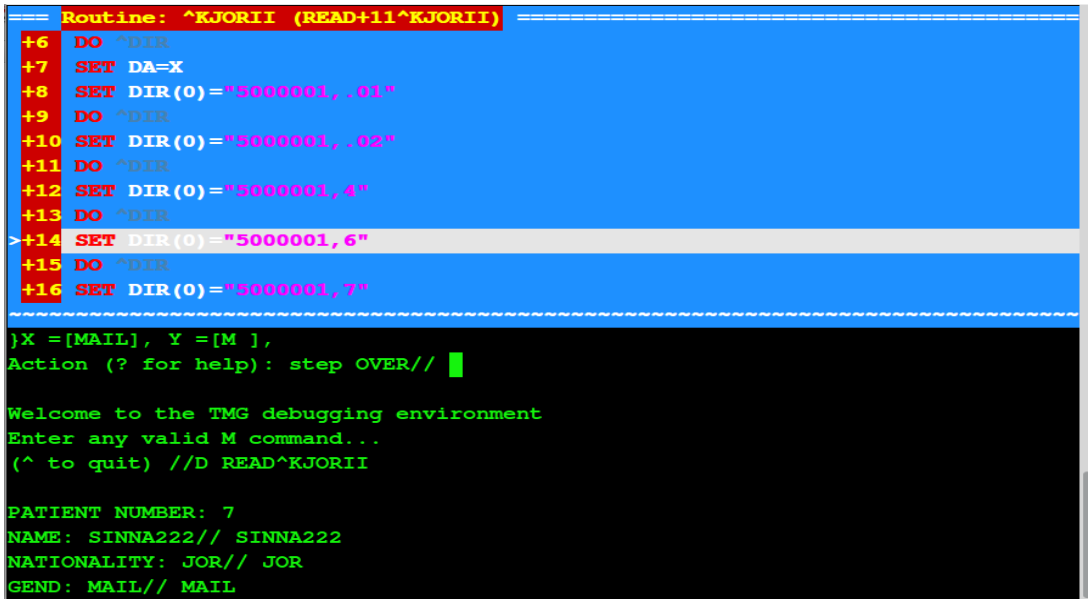
5.2 Debugging

5.2.1 Main Commands

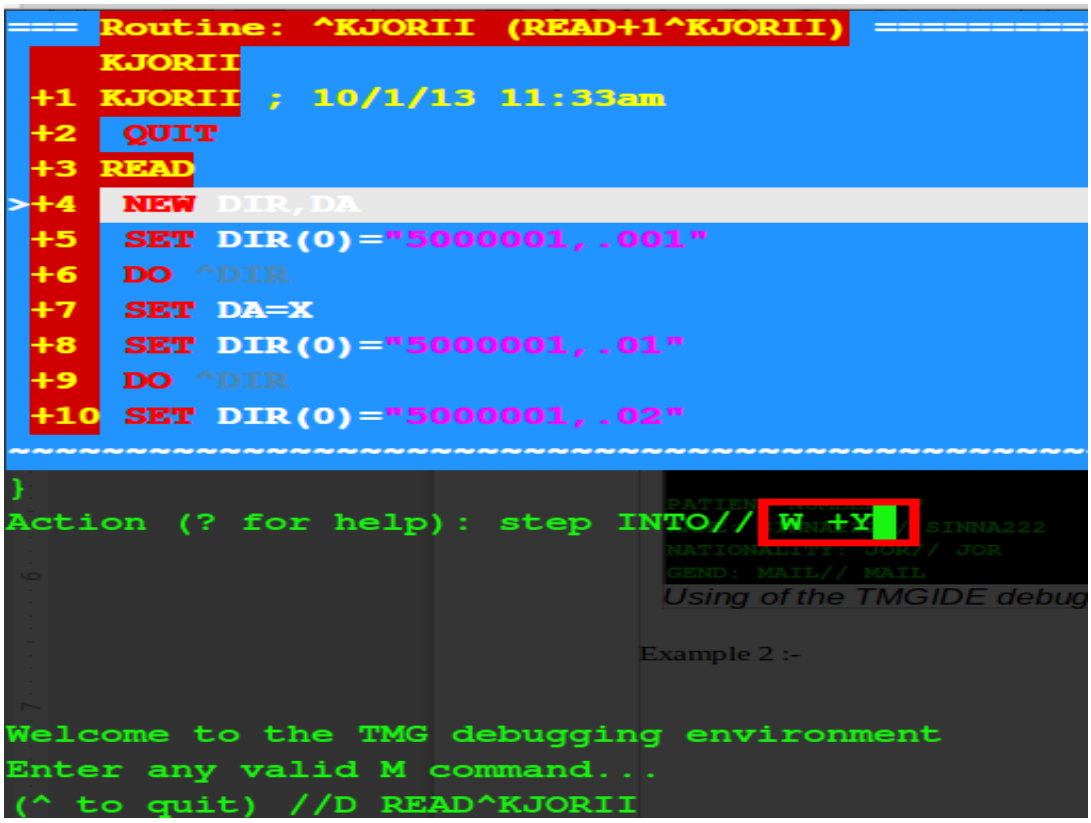
1. M : Execute M code
2. O : Step OVER line
3. I : Step INTO line
4. R : Run
5. T : Step OUT
6. CLS : Clear screen
7. W +MyVar :Watch MyVar
8. W -MyVar :Remove watch
9. SHOW [var] : Show [var]
10. Q : Abort

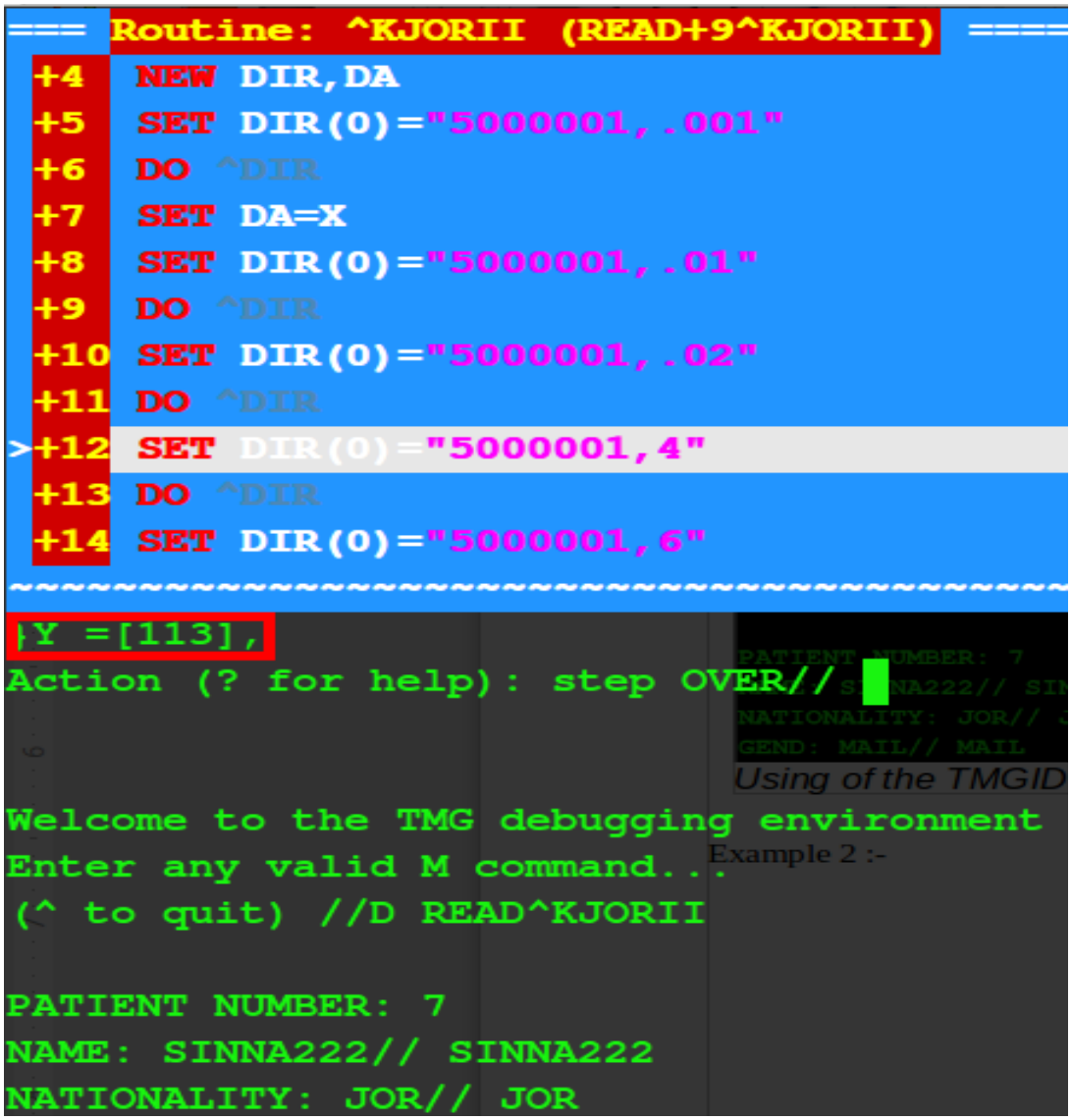
For more details about the previous commands refer to the following link:
http://tinco.pair.com/bhaskar/GT.M/doc/books/pg/UNIX_manual/index.html

Example 1:



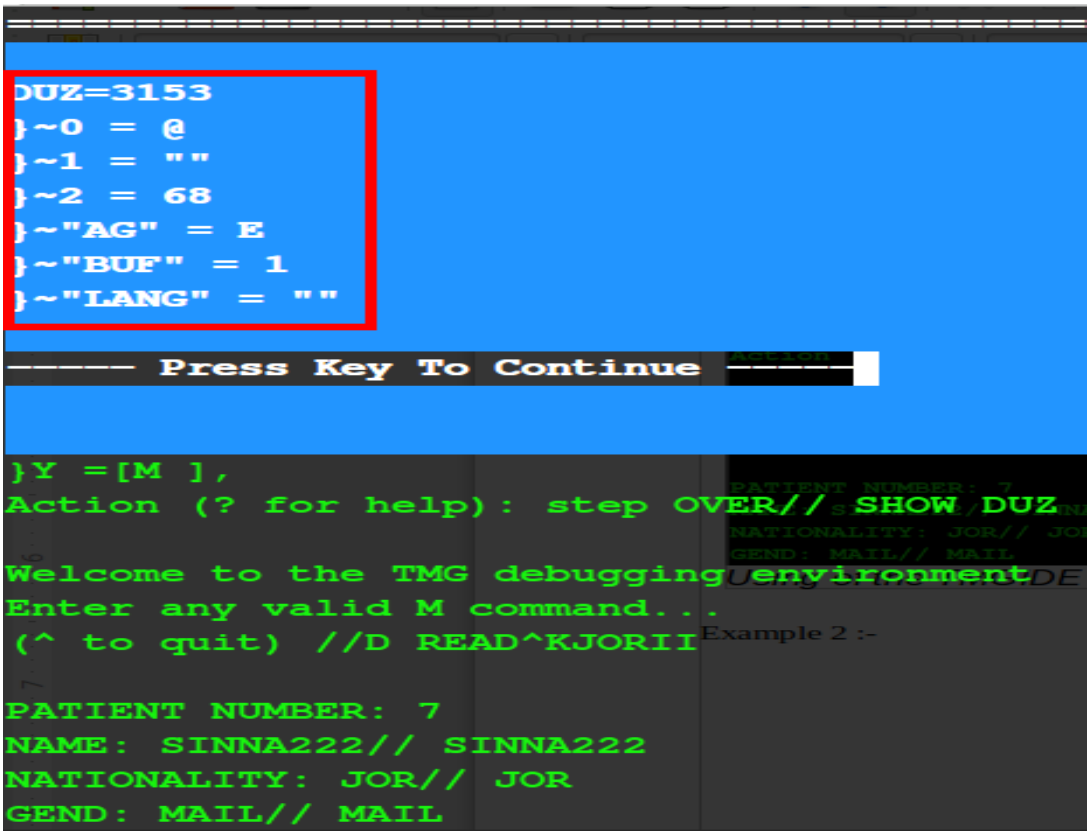
Example 2:





Using of the TMGIDE debu

Example 2 :-



Example 2 :-

6. References

1. http://tinco.pair.com/bhaskar/GT.M/doc/books/pg/UNIX_manual/index.html
2. www.va.gov/VISTA_MONOGRAPH/docs/2008_2009_VistAHealtheVet_Monograph_FC_0309.pdf